PW 01: Data Preparation

3- Grouping, joins, and aggregations

- **Objective:**: Master data summarization using grouping, pivot tables, and advanced aggregations, while learning to combine multiple datasets through merging and concatenation.

3.1- Averages, sums, counts by category

In [1]:
```
# import libraries
import pandas as pd

df = pd.read_csv ('cars.csv')
df.head()       # first 5 lines
```

Out[1]:

	Unnamed: 0	car_name	brand	model	vehicle_age	km_driven	seller_type	fuel_type	transmission_type	mileage	engine	max_power	seats	price
0	0	Maruti Alto	Maruti	Alto	9	120000	Individual	Petrol	Manual	19.70	796	46.30	5	120000
1	1	Hyundai Grand	Hyundai	Grand	5	20000	Individual	Petrol	Manual	18.90	1197	82.00	5	550000
2	2	Hyundai i20	Hyundai	i20	11	60000	Individual	Petrol	Manual	17.00	1197	80.00	5	215000
3	3	Maruti Alto	Maruti	Alto	9	37000	Individual	Petrol	Manual	20.92	998	67.10	5	226000
4	4	Ford Ecosport	Ford	Ecosport	6	30000	Dealer	Diesel	Manual	22.77	1498	98.59	5	570000

mean() – **Average of numeric columns**

In [12]:
```
df.groupby('brand').mean(numeric_only=True)
# This tells Pandas to group by the 'brand' column and t
# hen calculate the mean for all other numeric columns.
```

Out[12]:

brand	Unnamed: 0	vehicle_age	km_driven	mileage	engine	max_power	seats	price
Audi	10481.260417	7.369792	58611.531250	16.135990	2141.807292	181.426042	5.234375	1.966865e+06
BMW	10640.009112	6.619590	50934.384966	17.440182	2262.692483	207.939863	4.888383	2.693827e+06
Bentley	10281.666667	9.333333	25500.000000	8.033333	5998.000000	592.666667	4.333333	9.266667e+06
Datsun	8764.058824	4.117647	33003.576471	21.215647	1031.441176	62.645294	5.505882	3.205176e+05
Ferrari	4845.000000	2.000000	3800.000000	4.000000	3855.000000	601.000000	4.000000	3.950000e+07
Force	17068.000000	5.000000	60000.000000	17.000000	2596.000000	80.800000	5.000000	7.000000e+05
Ford	9850.230380	5.927848	58088.222785	19.922620	1504.440506	96.492873	5.118987	6.452241e+05
Honda	10222.248485	6.263973	53499.156229	19.908795	1448.259933	104.347152	4.997980	6.177569e+05
Hyundai	9896.910463	6.119048	50268.915493	19.588776	1316.671362	92.407250	5.000000	5.761539e+05
ISUZU	11636.000000	4.000000	56014.500000	13.800000	2999.000000	175.000000	7.000000	1.897500e+06
Isuzu	9129.375000	3.750000	76141.500000	12.575000	2561.500000	139.071250	5.250000	1.355000e+06
Jaguar	9791.864407	6.372881	44219.457627	16.089831	2309.508475	205.677458	5.000000	2.643034e+06
Jeep	9334.292683	3.195122	33401.365854	16.236585	1849.951220	172.375610	5.000000	1.795805e+06
Kia	9851.312500	1.656250	9255.437500	17.323125	1547.500000	130.025000	5.312500	1.735250e+06
Land Rover	10151.411765	6.078431	67092.901961	13.030392	2502.941176	209.007255	5.705882	3.823902e+06
Lexus	12703.200000	3.600000	28689.400000	20.846000	2683.200000	219.377000	5.000000	5.146500e+06
MG	8102.947368	1.684211	11538.684211	15.620526	1637.052632	150.829474	5.000000	1.752947e+06
Mahindra	9115.588526	6.018793	73182.502473	15.873778	2110.401583	109.005529	6.998022	7.874550e+05
Maruti	9666.381611	5.939103	51377.816707	22.430980	1141.906450	74.508480	5.145633	4.870893e+05
Maserati	7878.500000	5.000000	12250.000000	19.820000	2987.000000	270.940000	5.000000	6.100000e+06
Mercedes-AMG	12021.000000	4.000000	24000.000000	11.900000	2996.000000	362.070000	5.000000	5.100000e+06
Mercedes-Benz	10556.412463	7.623145	59322.175074	13.535757	2402.462908	210.906825	5.172107	2.480742e+06
Mini	9607.882353	6.352941	32210.705882	18.287647	1880.176471	151.167059	4.647059	2.182647e+06
Nissan	10403.090909	4.000000	37853.363636	16.151818	1604.818182	122.119091	4.545455	9.553636e+05
Porsche	8963.095238	6.904762	58356.952381	13.515714	3282.380952	299.028571	4.857143	5.161190e+06
Renault	9730.748134	4.837687	47705.733209	22.099142	1132.113806	74.901194	5.044776	4.409851e+05
Rolls-Royce	13944.000000	4.000000	5000.000000	10.200000	6592.000000	563.000000	4.000000	2.420000e+07
Skoda	10240.529940	6.835329	65488.730539	17.667006	1714.829341	126.014192	5.000000	7.840898e+05
Tata	9020.148837	4.283721	47828.413953	19.755279	1578.639535	109.933837	5.651163	6.835349e+05
Toyota	9888.443884	6.532156	91768.587642	13.214073	2557.187894	135.896419	7.118537	1.371317e+06
Volkswagen	10116.911290	6.675806	62697.804839	18.689774	1373.674194	92.552129	4.996774	5.165468e+05
Volvo	8758.300000	5.400000	61232.300000	14.860000	2122.200000	205.350000	6.100000	3.729700e+06

sum() – **Total of numeric columns**

In [4]:
```
df.groupby('fuel_type')['max_power'].sum()
```

Out[4]:
```
fuel_type
CNG           19466.31
Diesel      847974.64
Electric        630.80
LPG            2650.50
Petrol      679443.33
Name: max_power, dtype: float64
```

size() – **Number of rows per group (regardless of column values)**

In [5]:
```
df.groupby('fuel_type').size()
```

Out[5]:
```
fuel_type
CNG           301
Diesel       7419
Electric        4
LPG            44
Petrol       7643
dtype: int64
```

count() – **Counts non-null values in each column within each group**

In [6]:
```
df.groupby('brand').count()
```

Out[6]:

brand	Unnamed: 0	car_name	model	vehicle_age	km_driven	seller_type	fuel_type	transmission_type	mileage	engine	max_power	seats	price
Audi	192	192	192	192	192	192	192	192	192	192	192	192	192
BMW	439	439	439	439	439	439	439	439	439	439	439	439	439
Bentley	3	3	3	3	3	3	3	3	3	3	3	3	3
Datsun	170	170	170	170	170	170	170	170	170	170	170	170	170
Ferrari	1	1	1	1	1	1	1	1	1	1	1	1	1
Force	1	1	1	1	1	1	1	1	1	1	1	1	1
Ford	790	790	790	790	790	790	790	790	790	790	790	790	790
Honda	1485	1485	1485	1485	1485	1485	1485	1485	1485	1485	1485	1485	1485
Hyundai	2982	2982	2982	2982	2982	2982	2982	2982	2982	2982	2982	2982	2982
ISUZU	2	2	2	2	2	2	2	2	2	2	2	2	2
Isuzu	8	8	8	8	8	8	8	8	8	8	8	8	8
Jaguar	59	59	59	59	59	59	59	59	59	59	59	59	59
Jeep	41	41	41	41	41	41	41	41	41	41	41	41	41
Kia	32	32	32	32	32	32	32	32	32	32	32	32	32
Land Rover	51	51	51	51	51	51	51	51	51	51	51	51	51
Lexus	10	10	10	10	10	10	10	10	10	10	10	10	10
MG	19	19	19	19	19	19	19	19	19	19	19	19	19
Mahindra	1011	1011	1011	1011	1011	1011	1011	1011	1011	1011	1011	1011	1011
Maruti	4992	4992	4992	4992	4992	4992	4992	4992	4992	4992	4992	4992	4992
Maserati	2	2	2	2	2	2	2	2	2	2	2	2	2
Mercedes-AMG	1	1	1	1	1	1	1	1	1	1	1	1	1
Mercedes-Benz	337	337	337	337	337	337	337	337	337	337	337	337	337
Mini	17	17	17	17	17	17	17	17	17	17	17	17	17
Nissan	11	11	11	11	11	11	11	11	11	11	11	11	11
Porsche	21	21	21	21	21	21	21	21	21	21	21	21	21
Renault	536	536	536	536	536	536	536	536	536	536	536	536	536
Rolls-Royce	1	1	1	1	1	1	1	1	1	1	1	1	1
Skoda	334	334	334	334	334	334	334	334	334	334	334	334	334
Tata	430	430	430	430	430	430	430	430	430	430	430	430	430
Toyota	793	793	793	793	793	793	793	793	793	793	793	793	793
Volkswagen	620	620	620	620	620	620	620	620	620	620	620	620	620
Volvo	20	20	20	20	20	20	20	20	20	20	20	20	20

✏ GroupBy Aggregation Methods

Method	What it Does	Example Use Case
`mean()`	Calculates the **average** of numeric values	Average price per brand
`sum()`	Calculates the **total sum** of numeric values	Total power by fuel type
`size()`	Counts the **total number of rows** in each group	Number of cars per transmission type
`count()`	Counts **non-null values** in each column per group	Number of cars with known mileage per brand

3.2- Multiple aggregations with agg()

In [ ]:
```
# Aggregation: Aggregation is the process of combining multiple values into a single summary statistic, such as a sum, average, or count.
```

In [7]:
```
print(df.columns)
```

```
Index(['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven',
       'seller_type', 'fuel_type', 'transmission_type', 'mileage', 'engine',
       'max_power', 'seats', 'price'],
      dtype='object')
```

In [9]:
```
df.groupby('brand').agg({
      'price': 'mean',
      'engine': 'median'   # no spaces in the column name
})


# df.groupby('brand').agg({
#       'price': ['mean', 'max', 'std'],
#       'engine': 'median'   # no spaces in the column name
# })
```

Out[9]:

brand	price	engine
Audi	1.966865e+06	1968.0
BMW	2.693827e+06	1995.0
Bentley	9.266667e+06	5998.0
Datsun	3.205176e+05	1098.5
Ferrari	3.950000e+07	3855.0
Force	7.000000e+05	2596.0
Ford	6.452241e+05	1498.0
Honda	6.177569e+05	1497.0
Hyundai	5.761539e+05	1197.0
ISUZU	1.897500e+06	2999.0
Isuzu	1.355000e+06	2499.0
Jaguar	2.643034e+06	2179.0
Jeep	1.795805e+06	1956.0
Kia	1.735250e+06	1493.0
Land Rover	3.823902e+06	2179.0
Lexus	5.146500e+06	2487.0
MG	1.752947e+06	1451.0
Mahindra	7.874550e+05	2179.0
Maruti	4.870893e+05	1197.0
Maserati	6.100000e+06	2987.0
Mercedes-AMG	5.100000e+06	2996.0
Mercedes-Benz	2.480742e+06	2143.0
Mini	2.182647e+06	1998.0
Nissan	9.553636e+05	1498.0
Porsche	5.161190e+06	2967.0
Renault	4.409851e+05	999.0
Rolls-Royce	2.420000e+07	6592.0
Skoda	7.840898e+05	1598.0
Tata	6.835349e+05	1199.0
Toyota	1.371317e+06	2494.0
Volkswagen	5.165468e+05	1498.0
Volvo	3.729700e+06	1976.5

3.3- Aggregation across multiple columns simultaneously

In [11]:
```
# This will return the average price per combination of brand and fuel type (like Petrol, Diesel, etc.).
gb = df.groupby(['brand', 'fuel_type'])
gb['price'].mean()
```

Out[11]:
```
brand       fuel_type
Audi        Diesel        1.951424e+06
            Petrol        2.086182e+06
BMW         Diesel        2.715868e+06
            Petrol        2.495955e+06
Bentley     Petrol        9.266667e+06
                                ...
Toyota      Electric      1.853500e+06
            Petrol        1.360307e+06
Volkswagen Diesel         5.000651e+05
            Petrol        5.327125e+05
Volvo       Diesel        3.729700e+06
Name: price, Length: 61, dtype: float64
```

3.4- Pivot Tables

In [26]:
```
pd.pivot_table(
      df,
      values='price',
      index='brand',
      columns='fuel_type',
      aggfunc='mean'
)
```

Out[26]:

fuel_type brand	CNG	Diesel	Electric	LPG	Petrol
Audi	NaN	1.951424e+06	NaN	NaN	2.086182e+06
BMW	NaN	2.715868e+06	NaN	NaN	2.495955e+06
Bentley	NaN	NaN	NaN	NaN	9.266667e+06
Datsun	NaN	NaN	NaN	NaN	3.205176e+05
Ferrari	NaN	NaN	NaN	NaN	3.950000e+07
Force	NaN	7.000000e+05	NaN	NaN	NaN
Ford	NaN	6.785722e+05	NaN	NaN	5.731920e+05
Honda	422000.000000	6.191512e+05	NaN	NaN	6.175078e+05
Hyundai	435416.666667	6.946809e+05	NaN	247769.230769	5.134466e+05
ISUZU	NaN	1.897500e+06	NaN	NaN	NaN
Isuzu	NaN	1.355000e+06	NaN	NaN	NaN
Jaguar	NaN	2.594446e+06	NaN	NaN	3.550000e+06
Jeep	NaN	1.666731e+06	NaN	NaN	2.019533e+06
Kia	NaN	2.071357e+06	NaN	NaN	1.473833e+06
Land Rover	NaN	3.727688e+06	NaN	NaN	5.363333e+06
Lexus	NaN	NaN	NaN	NaN	5.146500e+06
MG	NaN	1.793429e+06	NaN	NaN	1.729333e+06
Mahindra	400000.000000	8.020753e+05	NaN	NaN	5.357963e+05
Maruti	414771.126761	6.144817e+05	NaN	188870.967742	4.244746e+05
Maserati	NaN	6.100000e+06	NaN	NaN	NaN
Mercedes-AMG	NaN	NaN	NaN	NaN	5.100000e+06
Mercedes-Benz	NaN	2.489936e+06	NaN	NaN	2.454727e+06
Mini	NaN	1.866429e+06	NaN	NaN	2.404000e+06
Nissan	NaN	9.698571e+05	NaN	NaN	9.300000e+05
Porsche	NaN	4.781000e+06	NaN	NaN	6.111667e+06
Renault	NaN	5.569234e+05	NaN	NaN	3.668838e+05
Rolls-Royce	NaN	NaN	NaN	NaN	2.420000e+07
Skoda	NaN	7.650153e+05	NaN	NaN	8.111812e+05
Tata	NaN	7.917804e+05	NaN	NaN	5.258057e+05
Toyota	730000.000000	1.371569e+06	1853500.0	NaN	1.360307e+06
Volkswagen	NaN	5.000651e+05	NaN	NaN	5.327125e+05
Volvo	NaN	3.729700e+06	NaN	NaN	NaN

In [28]:
```
# To fill missing values (e.g., if a brand doesn't offer a fuel type):

pd.pivot_table(
      df,
      values='price',
      index='brand',
      columns='fuel_type',
      aggfunc='mean',
      fill_value=0
)
```

Out[28]:

fuel_type brand	CNG	Diesel	Electric	LPG	Petrol
Audi	0.000000	1.951424e+06	0.0	0.000000	2.086182e+06
BMW	0.000000	2.715868e+06	0.0	0.000000	2.495955e+06
Bentley	0.000000	0.000000e+00	0.0	0.000000	9.266667e+06
Datsun	0.000000	0.000000e+00	0.0	0.000000	3.205176e+05
Ferrari	0.000000	0.000000e+00	0.0	0.000000	3.950000e+07
Force	0.000000	7.000000e+05	0.0	0.000000	0.000000e+00
Ford	0.000000	6.785722e+05	0.0	0.000000	5.731920e+05
Honda	422000.000000	6.191512e+05	0.0	0.000000	6.175078e+05
Hyundai	435416.666667	6.946809e+05	0.0	247769.230769	5.134466e+05
ISUZU	0.000000	1.897500e+06	0.0	0.000000	0.000000e+00
Isuzu	0.000000	1.355000e+06	0.0	0.000000	0.000000e+00
Jaguar	0.000000	2.594446e+06	0.0	0.000000	3.550000e+06
Jeep	0.000000	1.666731e+06	0.0	0.000000	2.019533e+06
Kia	0.000000	2.071357e+06	0.0	0.000000	1.473833e+06
Land Rover	0.000000	3.727688e+06	0.0	0.000000	5.363333e+06
Lexus	0.000000	0.000000e+00	0.0	0.000000	5.146500e+06
MG	0.000000	1.793429e+06	0.0	0.000000	1.729333e+06
Mahindra	400000.000000	8.020753e+05	0.0	0.000000	5.357963e+05
Maruti	414771.126761	6.144817e+05	0.0	188870.967742	4.244746e+05
Maserati	0.000000	6.100000e+06	0.0	0.000000	0.000000e+00
Mercedes-AMG	0.000000	0.000000e+00	0.0	0.000000	5.100000e+06
Mercedes-Benz	0.000000	2.489936e+06	0.0	0.000000	2.454727e+06
Mini	0.000000	1.866429e+06	0.0	0.000000	2.404000e+06
Nissan	0.000000	9.698571e+05	0.0	0.000000	9.300000e+05
Porsche	0.000000	4.781000e+06	0.0	0.000000	6.111667e+06
Renault	0.000000	5.569234e+05	0.0	0.000000	3.668838e+05
Rolls-Royce	0.000000	0.000000e+00	0.0	0.000000	2.420000e+07
Skoda	0.000000	7.650153e+05	0.0	0.000000	8.111812e+05
Tata	0.000000	7.917804e+05	0.0	0.000000	5.258057e+05
Toyota	730000.000000	1.371569e+06	1853500.0	0.000000	1.360307e+06
Volkswagen	0.000000	5.000651e+05	0.0	0.000000	5.327125e+05
Volvo	0.000000	3.729700e+06	0.0	0.000000	0.000000e+00

In [13]:

```
# Pivot Table: A pivot table is a data summarization tool that reorganizes and groups data into a grid format,
# allowing you to calculate aggregations across both rows and columns simultaneously.


# To round the results:

pivot = pd.pivot_table(df,
                       values='price',
                       index='brand',
                       columns='fuel_type',
                       aggfunc='mean')
pivot
# pivot.round(0)
```

Out[13]:

fuel_type brand	CNG	Diesel	Electric	LPG	Petrol
Audi	NaN	1951424.0	NaN	NaN	2086182.0
BMW	NaN	2715868.0	NaN	NaN	2495955.0
Bentley	NaN	NaN	NaN	NaN	9266667.0
Datsun	NaN	NaN	NaN	NaN	320518.0
Ferrari	NaN	NaN	NaN	NaN	39500000.0
Force	NaN	700000.0	NaN	NaN	NaN
Ford	NaN	678572.0	NaN	NaN	573192.0
Honda	422000.0	619151.0	NaN	NaN	617508.0
Hyundai	435417.0	694681.0	NaN	247769.0	513447.0
ISUZU	NaN	1897500.0	NaN	NaN	NaN
Isuzu	NaN	1355000.0	NaN	NaN	NaN
Jaguar	NaN	2594446.0	NaN	NaN	3550000.0
Jeep	NaN	1666731.0	NaN	NaN	2019533.0
Kia	NaN	2071357.0	NaN	NaN	1473833.0
Land Rover	NaN	3727688.0	NaN	NaN	5363333.0
Lexus	NaN	NaN	NaN	NaN	5146500.0
MG	NaN	1793429.0	NaN	NaN	1729333.0
Mahindra	400000.0	802075.0	NaN	NaN	535796.0
Maruti	414771.0	614482.0	NaN	188871.0	424475.0
Maserati	NaN	6100000.0	NaN	NaN	NaN
Mercedes-AMG	NaN	NaN	NaN	NaN	5100000.0
Mercedes-Benz	NaN	2489936.0	NaN	NaN	2454727.0
Mini	NaN	1866429.0	NaN	NaN	2404000.0
Nissan	NaN	969857.0	NaN	NaN	930000.0
Porsche	NaN	4781000.0	NaN	NaN	6111667.0
Renault	NaN	556923.0	NaN	NaN	366884.0
Rolls-Royce	NaN	NaN	NaN	NaN	24200000.0
Skoda	NaN	765015.0	NaN	NaN	811181.0
Tata	NaN	791780.0	NaN	NaN	525806.0
Toyota	730000.0	1371569.0	1853500.0	NaN	1360307.0
Volkswagen	NaN	500065.0	NaN	NaN	532712.0
Volvo	NaN	3729700.0	NaN	NaN	NaN

3.5- Sorting aggregated groups

In [32]:

```
df.groupby('brand')['price'].mean().sort_values(ascending = False)
```

Out[32]:
```
brand
Ferrari         3.950000e+07
Rolls-Royce     2.420000e+07
Bentley         9.266667e+06
Maserati        6.100000e+06
Porsche         5.161190e+06
Lexus           5.146500e+06
Mercedes-AMG    5.100000e+06
Land Rover      3.823902e+06
Volvo           3.729700e+06
BMW             2.693827e+06
Jaguar          2.643034e+06
Mercedes-Benz   2.480742e+06
Mini            2.182647e+06
Audi            1.966865e+06
ISUZU
                1.897500e+06
Jeep              1.795805e+06
MG                1.752947e+06
Kia               1.735250e+06
Toyota            1.371317e+06
Isuzu             1.355000e+06
Nissan            9.553636e+05
Mahindra          7.874550e+05
Skoda             7.840898e+05
Force             7.000000e+05
Tata              6.835349e+05
Ford              6.452241e+05
Honda             6.177569e+05
Hyundai           5.761539e+05
Volkswagen        5.165468e+05
Maruti            4.870893e+05
Renault           4.409851e+05
Datsun            3.205176e+05
Name: price, dtype: float64
```

3.6- Merges between DataFrames (relational joins)

In [41]:
```python
# Re-import libraries after code execution reset
import pandas as pd

# Reload the original cars.csv dataset
df_cars = pd.read_csv('cars.csv')

# Simulated second DataFrame for merging demonstration
df_extra = pd.DataFrame({
      'brand': ['Maruti', 'Hyundai', 'Ford', 'BMW'],
      'avg_service_cost': [5000, 7000, 8000, 12000]
})

# Perform the merge
merged_df = pd.merge(df_cars, df_extra, on='brand', how='inner')

# how='inner' acts like an Intersection.

# It tells Pandas: "Only keep the rows where the key (brand) exists in BOTH DataFrames."
```

```
merged_df.head()
```

Out[41]:

	Unnamed: 0	car_name	brand	model	vehicle_age	km_driven	seller_type	fuel_type	transmission_type	mileage	engine	max_power	seats	price	avg_service_cost
0	0	Maruti Alto	Maruti	Alto	9	120000	Individual	Petrol	Manual	19.70	796	46.30	5	120000	5000
1	3	Maruti Alto	Maruti	Alto	9	37000	Individual	Petrol	Manual	20.92	998	67.10	5	226000	5000
2	5	Maruti Wagon R	Maruti	Wagon R	8	35000	Individual	Petrol	Manual	18.90	998	67.10	5	350000	5000
3	7	Maruti Wagon R	Maruti	Wagon R	3	17512	Dealer	Petrol	Manual	20.51	998	67.04	5	410000	5000
4	12	Maruti Swift	Maruti	Swift	4	28321	Dealer	Petrol	Manual	16.60	1197	85.00	5	511000	5000

In [42]:

```
# Re-import libraries after code execution reset
import pandas as pd

# Reload the original cars.csv dataset
df_cars = pd.read_csv('cars.csv')

# Simulated second DataFrame with a different column name for 'brand'
df_extra = pd.DataFrame({
     'car_brand': ['Maruti', 'Hyundai', 'Ford', 'BMW'], # renamed column
     'avg_service_cost': [5000, 7000, 8000, 12000]
})

# Perform the merge using left_on and right_on
merged_df = pd.merge(df_cars, df_extra, left_on='brand', right_on='car_brand', how='inner')

# Show the first few rows
print(merged_df.head())
```

```
   Unnamed: 0         car_name  brand     model vehicle_age km_driven \
0           0      Maruti Alto Maruti      Alto           9   120000
1           3      Maruti Alto Maruti      Alto           9    37000
2           5 Maruti Wagon R Maruti Wagon R               8    35000
3           7 Maruti Wagon R Maruti Wagon R               3    17512
4          12     Maruti Swift Maruti     Swift           4    28321

 seller_type fuel_type transmission_type mileage engine max_power seats \
0 Individual       Petrol           Manual     19.70    796     46.30     5
1 Individual       Petrol           Manual     20.92    998     67.10     5
2 Individual       Petrol           Manual     18.90    998     67.10     5
3       Dealer     Petrol           Manual     20.51    998     67.04     5
4       Dealer     Petrol           Manual     16.60   1197     85.00     5

     price car_brand avg_service_cost
0 120000       Maruti             5000
1 226000       Maruti             5000
2 350000       Maruti             5000
3 410000       Maruti             5000
4 511000       Maruti             5000
```

✏ **Summary of join types:**

how	Description
`'inner'`	Only matching rows
`'left'`	All rows from the left DataFrame + matches from right
`'right'`	All rows from the right DataFrame + matches from left
`'outer'`	All rows from both DataFrames

3.7- Concatenation (stacking multiple arrays)

In [ ]:

```
pd.concat([ df1, df2 ], axis = 0)              # line by line
```

In [14]:

```
import pandas as pd
```

```
# Load your cars.csv file
df_cars = pd.read_csv('cars.csv')

# Split and concatenate
df1 = df_cars.iloc[:5]
df2 = df_cars.iloc[5:10]

# Row-wise concat (stack rows)
concat_rows = pd.concat([df1, df2], axis=0)
print("Row-wise concatenation:")
print(concat_rows.head(10))
```

```
Row-wise concatenation:
   Unnamed: 0        car_name    brand     model vehicle_age km_driven \
0           0     Maruti Alto   Maruti      Alto           9     120000
1           1   Hyundai Grand Hyundai      Grand           5      20000
2           2     Hyundai i20 Hyundai        i20          11      60000
3           3     Maruti Alto   Maruti      Alto           9      37000
4           4   Ford Ecosport     Ford Ecosport            6      30000
5           5 Maruti Wagon R    Maruti   Wagon R           8      35000
6           6     Hyundai i10 Hyundai        i10           8      40000
7
         7 Maruti Wagon R   Maruti    Wagon R              3      17512
8            8   Hyundai Venue Hyundai      Venue              2      20000
9           12    Maruti Swift  Maruti      Swift              4      28321

 seller_type fuel_type transmission_type mileage      engine   max_power   seats   \
0 Individual      Petrol           Manual     19.70      796       46.30       5
1 Individual      Petrol           Manual     18.90     1197       82.00       5
2 Individual      Petrol           Manual     17.00     1197       80.00       5
3 Individual      Petrol           Manual     20.92      998       67.10       5
4       Dealer    Diesel           Manual     22.77     1498       98.59       5
5 Individual      Petrol           Manual     18.90      998       67.10       5
6       Dealer    Petrol           Manual     20.36     1197       78.90       5
7       Dealer    Petrol           Manual     20.51      998       67.04       5
8 Individual      Petrol        Automatic     18.15      998      118.35       5
9       Dealer    Petrol           Manual     16.60     1197       85.00       5

      price
0    120000
1    550000
2    215000
3    226000
4    570000
5    350000
6    315000
7    410000
8 1050000
9    511000
```

In [ ]: